

Funzioni lambda

Funzioni lambda

Funzioni piccole e veloci da scrivere in una sola riga.

Cosa sono le funzioni lambda?

Quando hai bisogno di una funzione semplice, che fa solo una cosa, e non vuoi sprecare righe di codice per definirla con `def`, puoi usare una **funzione lambda**.

È come una funzione normale, ma compressa in una singola riga, senza nome e senza `return` esplicito.

La forma base

```
lambda parametri: espressione
```

Confronta una funzione normale con la sua versione lambda:

```
# Funzione normale
def raddoppia(x):
    return x * 2

# La stessa cosa come lambda
raddoppia_lambda = lambda x: x * 2

print(raddoppia(5))          # 10
print(raddoppia_lambda(5))   # 10
```

Puoi anche avere più parametri:

```
somma = lambda a, b: a + b
print(somma(3, 5)) # 8
```

Quando usare le lambda?

Le lambda sono utili quando devi passare una "piccola funzione" come argomento a un'altra funzione. Il caso più comune è con `sorted()`, `map()` e `filter()`.

Ordinare una lista di oggetti complessi

Se hai una lista di tuple (coppie di valori) e vuoi ordinarla in base al secondo elemento, la lambda è perfetta:

```
studenti = [
    ("Alice", 16),
    ("Bob", 15),
    ("Carlo", 17)
]

# Ordina per età (il secondo elemento di ogni tupla)
ordinati = sorted(studenti, key=lambda s: s[1])
print(ordinati)
# [('Bob', 15), ('Alice', 16), ('Carlo', 17)]
```

Trasformare ogni elemento di una lista con `map()`

`map()` applica una funzione a ogni elemento di una sequenza:

```
numeri = [1, 2, 3, 4, 5]
quadrati = list(map(lambda x: x ** 2, numeri))
print(quadrati) # [1, 4, 9, 16, 25]
```

Equivale a: "per ogni numero nella lista, calcola il suo quadrato".

Filtrare elementi con `filter()`

`filter()` tiene solo gli elementi per cui la funzione restituisce `True` :

```
numeri = [1, 2, 3, 4, 5, 6, 7, 8]
pari = list(filter(lambda x: x % 2 == 0, numeri))
print(pari) # [2, 4, 6, 8]
```

Equivale a: "tieni solo i numeri che, divisi per 2, danno resto zero".

Limiti delle lambda

Le lambda sono semplici per design: possono contenere solo **una singola espressione**. Non puoi usare `if...elif...else` su più righe, cicli, o più istruzioni.

Quando la logica diventa più complessa di una riga, usa sempre una funzione normale con `def` :

```
# Questo va bene con lambda:
doppio = lambda x: x * 2

# Questo è troppo complesso – usa def:
def classifica(voto):
    if voto >= 9:
        return "Ottimo"
    elif voto >= 6:
        return "Sufficiente"
    else:
        return "Insufficiente"
```